

Today 6:32 PM

I want to know everything about apex testing... 1 unit testing, 2 testing triggers, test data and salesforce data

Explain All the concept and what exactly are they in simple way and with examples also when to use them and how to use them...

Step by step how to use them as in how to write the codes, line by line explain the codes we write regarding that... why do we write them...

give few examples as in qsns and write the actual code and testing code and mock testing code for them and

Show more

Absolutely. Since you're learning Apex for both **practical Salesforce work and interviews**, the best way is to build the concept from the ground up rather than memorizing test-class syntax.

The four things you mentioned are closely connected:

1. **Apex Unit Testing**
2. **Testing Apex Triggers**
3. **Test Data**
4. **Testing Salesforce data / database behavior**
5. **Mock Testing** — especially important for REST/SOAP callouts

I'll also show you the **actual Apex code** → **test code** → **mock test code**, and explain the important lines.

Salesforce requires Apex tests for deployment/package scenarios, and Apex classes/triggers need sufficient coverage; Salesforce's documented minimum is **75% coverage**, but good tests should test actual behavior and edge cases rather than simply chasing the percentage.  Developer +1

1. First: What exactly is Apex Testing?

Suppose you write:

```
apex

public class Calculator {

    public static Integer add(Integer a, Integer b) {
        return a + b;
    }
}
```

You *think* this works.

But how do you prove it?

You write a test:

```
apex

@Test
private class CalculatorTest {

    @Test
    static void testAdd() {

        Integer result = Calculator.add(10, 20);

        System.assertEquals(30, result);
    }
}
```

The test is basically saying:

"Salesforce, run my code with these inputs and check whether I get the expected result."

So:

Production code

```
Does the application work?
    ↓
Apex class / Trigger
    ↓
```

Test code

Does the application actually behave correctly?

↓

Test class

↓

Assertions

2. What is Unit Testing?

A **unit test** tests one small piece of functionality independently.

For example:

apex

```
Calculator.add()
```

is one unit of functionality.

You test:

```
10 + 20
```

↓

```
30
```

rather than testing your entire Salesforce application at once.

Real-life analogy

Imagine a car.

You don't only test:

"Does the entire car drive?"

You separately test:

- brakes
- accelerator
- headlights
- horn
- engine
- steering

Those are like **units**.

In Apex:

```
Apex class method
    ↓
Unit
    ↓
Unit test
```

3. Basic structure of an Apex Test Class

Memorize this structure first:

```
apex

@Test
private class MyTestClass {

    @Test
    static void myTestMethod() {

        // Arrange
        // Create test data

        // Act
        // Execute the code

        // Assert
        // Verify result
    }
}
```

There are three major parts:

Arrange

Prepare everything.

```
apex

Integer a = 10;
Integer b = 20;
```

Act

Run the code.

apex

```
Integer result = Calculator.add(a, b);
```

Assert

Check what happened.

apex

```
System.assertEquals(30, result);
```

This is extremely important for interviews:

Arrange → Act → Assert

4. Understanding `@isTest`

You will see:

apex

```
@isTest
```

everywhere.

It tells Salesforce:

"This is test code, not normal application code."

For example:

apex

```
@isTest
private class CalculatorTest {
```

This means the entire class is a test class.

You can also put it on a method:

apex

```
@isTest
static void testAdd() {
```

Salesforce documentation recommends that classes marked `@isTest` contain test methods and supporting test functionality. Test code itself isn't counted toward the normal Apex code-size limit.  Develo...

5. `System.assertEquals()`

This is one of the most important things in Apex testing.

Suppose:

apex

```
Integer result = Calculator.add(10, 20);
```

We expect:

30

So:

apex

```
System.assertEquals(30, result);
```

means:

```
Expected = 30  
Actual   = result
```

If result = 30:

PASS

If result = 25:

FAIL

6. Three important assertions

```
System.assertEquals()
```

Use when you know exactly what the value should be.

apex

```
System.assertEquals(30, result);
```

```
System.assertNotEquals()
```

Use when something should NOT equal a particular value.

apex

```
System.assertNotEquals(50, result);
```

```
System.assert()
```

Use for true/false conditions.

apex

```
System.assert(result > 0);
```

Meaning:

| result must be greater than 0.

You can also provide a message:

apex

```
System.assertEquals(  
    30,  
    result,  
    'Addition should return 30'  
);
```

If it fails, the message helps you understand why.

7. First complete example

Let's say the interviewer asks:

| Create an Apex class that calculates the square of a number and write a test class.

Production code

apex

```
public class Calculator {  
  
    public static Integer square(Integer number) {  
        return number * number;  
    }  
}
```

Line by line

apex

```
public class Calculator {
```

Creates an Apex class called `Calculator`.

apex

```
public static Integer square(Integer number) {
```

Creates a method.

- `public` → accessible
 - `static` → call it using the class name
 - `Integer` → returns an Integer
 - `square` → method name
 - `Integer number` → input
-

apex

```
return number * number;
```

Returns square.

For:

5

it returns:

Test class

```
apex

@isTest
private class CalculatorTest {

    @isTest
    static void testSquare() {

        Integer result = Calculator.square(5);

        System.assertEquals(25, result);
    }
}
```

What happens?

```
Calculator.square(5)
    ↓
25
    ↓
result = 25
    ↓
assert 25 == 25
    ↓
PASS
```

8. But Salesforce is not just normal Java

This is where Apex testing becomes more interesting.

Salesforce code usually works with:

```
Account
Contact
Opportunity
Lead
Case
Custom Objects
etc.
```

So we need to test **Salesforce records** too.

For example:

apex

```
Account acc = new Account(  
    Name = 'Test Account'  
);  
  
insert acc;
```

Now you're testing database behavior.

9. What is Test Data?

Test data means:

Records created specifically for your test.

Example:

apex

```
Account acc = new Account(  
    Name = 'Test Account'  
);  
  
insert acc;
```

This creates an Account for testing.

You aren't supposed to depend on somebody's real Account in the org.

Instead:

```
Test  
↓  
Create its own data  
↓  
Run code  
↓  
Check result
```

Salesforce's testing guidance recommends creating test data rather than relying on existing organization data. Using `SeeAllData=true` should generally be avoided.

10. Why don't we simply query existing data?

You might think:

apex

```
Account acc = [  
    SELECT Id, Name  
    FROM Account  
    LIMIT 1  
];
```

Why not?

Because normal Apex tests generally run with access to isolated test data rather than your organization's existing business records.

That's actually a **good thing**.

Imagine your test depends on:

```
Account = ABC Company
```

Tomorrow someone deletes ABC Company.

Your test fails.

Instead:

apex

```
Account acc = new Account(Name = 'Test Account');  
insert acc;
```

Now the test controls its own data.

11. SeeAllData=true

You may see:

apex

```
@isTest(SeeAllData=true)
```

This allows the test to access existing organization data.

But don't make this your default approach.

Prefer:

```
apex
```

```
@isTest
```

and create your own data.

Interview question

Q: Why should we avoid SeeAllData=true?

Answer:

Because tests should be isolated and independent of existing organization data. Creating test data makes tests more reliable and portable.

12. Testing Salesforce Database Operations

Suppose we have this class:

```
apex
```

```
public class AccountService {  
  
    public static Account createAccount(String name) {  
  
        Account acc = new Account(  
            Name = name  
        );  
  
        insert acc;  
  
        return acc;  
    }  
}
```

Now let's test it.

```
apex
```

```
@isTest  
private class AccountServiceTest {  
  
    @isTest
```

```
static void testCreateAccount() {  
  
    Account acc = AccountService.createAccount('Test Company');  
  
    System.assertNotEquals(null, acc.Id);  
    System.assertEquals('Test Company', acc.Name);  
}  
}
```

13. Why check Id ?

Before insertion:

```
apex  
  
Account acc = new Account(  
    Name = 'Test Company'  
);
```

The record doesn't yet have a Salesforce Id.

After:

```
apex  
  
insert acc;
```

Salesforce assigns an Id.

So:

```
apex  
  
System.assertNotEquals(null, acc.Id);
```

means:

The record should actually have been inserted.

14. Testing using SOQL

Often you should query the database after performing an operation.

Example:

apex

```
@isTest
private class AccountServiceTest {

    @isTest
    static void testCreateAccount() {

        AccountService.createAccount('Test Company');

        Account acc = [
            SELECT Id, Name
            FROM Account
            WHERE Name = 'Test Company'
            LIMIT 1
        ];

        System.assertEquals('Test Company', acc.Name);
    }
}
```

Notice the flow:

```
Call method
↓
Method inserts Account
↓
SOQL retrieves Account
↓
Assert checks database result
```

This is very common in real Salesforce tests.

15. `Test.startTest()` and `Test.stopTest()`

These are extremely important.

You will see:

apex

```
Test.startTest();

// Code being tested

Test.stopTest();
```

Think of them as:

"Start the actual testing operation here."

and

"End the special test execution section here."

Example:

apex

```
@isTest
static void testCreateAccount() {

    Account acc = new Account(Name = 'Test');

    Test.startTest();

    insert acc;

    Test.stopTest();

    System.assertNotEquals(null, acc.Id);
}
```

16. Why use startTest/stopTest?

Two major reasons.

Reason 1 — Governor limits

It gives the code being tested a fresh set of governor-limit context.

Reason 2 — Asynchronous Apex

It allows asynchronous work such as:

- Future methods
- Queueable Apex
- Batch Apex
- Scheduled Apex

to execute during testing when appropriate.

Salesforce specifically documents `Test.startTest()` / `Test.stopTest()` as important for controlling test execution and asynchronous behavior.  Developer +1

17. Where should I put them?

A common pattern is:

apex

```
// Create data

Test.startTest();

// Execute functionality

Test.stopTest();

// Verify result
```

Example:

apex

```
@isTest
static void testCreateAccount() {

    Account acc = new Account(
        Name = 'Test Company'
    );

    Test.startTest();

    insert acc;

    Test.stopTest();

    System.assertNotEquals(null, acc.Id);
}
```

Don't blindly put every single line inside `startTest()` .

Usually:

```
Setup
↓
startTest()
↓
Actual code being tested
↓
stopTest()
↓
Assertions
```

18. @TestSetup

Now imagine you have five test methods.

Every test needs:

```
Account
Contact
Opportunity
```

You could recreate them five times.

That's inefficient.

Instead:

```
apex

@testSetup
static void setupData() {

    Account acc = new Account(
        Name = 'Test Account'
    );

    insert acc;
}
```

Then every test method can use that setup data.

Example:

```
apex

@isTest
private class AccountTest {

    @testSetup
    static void setupData() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        insert acc;
    }

    @isTest
```

```

static void testOne() {

    Account acc = [
        SELECT Id, Name
        FROM Account
        WHERE Name = 'Test Account'
        LIMIT 1
    ];

    System.assertEquals(
        'Test Account',
        acc.Name
    );
}

@Test
static void testTwo() {

    Account acc = [
        SELECT Id, Name
        FROM Account
        WHERE Name = 'Test Account'
        LIMIT 1
    ];

    System.assertNotEquals(
        null,
        acc.Id
    );
}
}

```

Salesforce recommends `@testSetup` for reusable test data across test methods.

 Develo...

19. Test Data Factory

As projects become larger, you don't want this everywhere:

apex

```

Account acc = new Account(
    Name = 'Test Account'
);

insert acc;

```

Instead you can create a helper:

apex

```
@isTest
public class TestDataFactory {

    public static Account createAccount() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        insert acc;

        return acc;
    }
}
```

Then:

apex

```
Account acc = TestDataFactory.createAccount();
```

This is called a **Test Data Factory**.

Why?

Because it centralizes test-data creation.

20. Now let's move to Trigger Testing

This is one of the most important interview topics.

Suppose we have:

Whenever an Account is created, automatically create a Task for that Account.

Trigger:

apex

```
trigger AccountTrigger on Account (after insert) {

    List<Task> tasks = new List<Task>();

    for (Account acc : Trigger.new) {
```

```

        Task t = new Task(
            Subject = 'Follow up',
            WhatId = acc.Id
        );

        tasks.add(t);
    }

    insert tasks;
}

```

21. What is happening?

Account gets inserted:

```

insert Account
    ↓
AccountTrigger fires
    ↓
after insert
    ↓
Trigger.new
    ↓
Create Task
    ↓
insert Task

```

Now we need a test.

22. Trigger test

```

apex

@isTest
private class AccountTriggerTest {

    @isTest
    static void testTaskCreation() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        Test.startTest();

        insert acc;
    }
}

```

```

Test.stopTest();

Task t = [
    SELECT Id, Subject, WhatId
    FROM Task
    WHERE WhatId = :acc.Id
    LIMIT 1
];

System.assertEquals(
    'Follow up',
    t.Subject
);

System.assertEquals(
    acc.Id,
    t.WhatId
);
}
}

```

23. This is the most important trigger-testing concept

Notice:

We don't call the trigger manually.

We don't write:

apex

```
AccountTrigger.run();
```

No.

We perform the DML:

apex

```
insert acc;
```

Salesforce automatically fires the trigger.

Therefore:

```
insert acc
↓
```

Trigger fires automatically

This is how you test triggers.

24. Testing before insert

Suppose:

```
apex

trigger AccountTrigger on Account (before insert) {

    for (Account acc : Trigger.new) {

        if (acc.Name == null) {
            acc.Name = 'Default Account';
        }
    }
}
```

Test:

```
apex

@isTest
static void testDefaultName() {

    Account acc = new Account();

    Test.startTest();

    insert acc;

    Test.stopTest();

    System.assertEquals(
        'Default Account',
        acc.Name
    );
}
```

Because this is before insert , the trigger modifies the record before Salesforce saves it.

25. Testing before update

Suppose:

apex

```
trigger AccountTrigger on Account (before update) {

    for (Account acc : Trigger.new) {

        if (acc.Industry == 'Technology') {
            acc.Description = 'Technology Company';
        }
    }
}
```

Test:

apex

```
@isTest
static void testBeforeUpdate() {

    Account acc = new Account(
        Name = 'Test Account'
    );

    insert acc;

    acc.Industry = 'Technology';

    Test.startTest();

    update acc;

    Test.stopTest();

    System.assertEquals(
        'Technology Company',
        acc.Description
    );
}
```

26. Testing after insert

Suppose:

apex

```
trigger AccountTrigger on Account (after insert) {

    for (Account acc : Trigger.new) {
```

```
        System.debug(acc.Id);
    }
}
```

Since `after insert` happens after the record has been saved, `Id` is available.

Your test:

apex

```
Account acc = new Account(
    Name = 'Test'
);

Test.startTest();

insert acc;

Test.stopTest();

System.assertNotEquals(null, acc.Id);
```

27. Testing `after update`

Same principle.

apex

```
Account acc = new Account(
    Name = 'Test'
);

insert acc;

acc.Name = 'Updated Test';

Test.startTest();

update acc;

Test.stopTest();

Account result = [
    SELECT Name
    FROM Account
    WHERE Id = :acc.Id
];
```

```
System.assertEquals(  
    'Updated Test',  
    result.Name  
);
```

28. Trigger.new and Trigger.old

This is extremely important for trigger tests.

Suppose trigger:

```
apex  
  
trigger AccountTrigger on Account (before update) {  
  
    for (Account acc : Trigger.new) {  
  
        Account oldAcc = Trigger.oldMap.get(acc.Id);  
  
        if (oldAcc.Name != acc.Name) {  
            acc.Description = 'Name Changed';  
        }  
    }  
}
```

The test must actually change the field:

```
apex  
  
acc.Name = 'New Name';  
  
update acc;
```

Otherwise:

```
Old Name = Test  
New Name = Test
```

Condition:

```
apex  
  
oldAcc.Name != acc.Name
```

is false.

29. Bulk Trigger Testing

This is a BIG interview topic.

Salesforce triggers are designed to handle multiple records.

Bad test:

apex

```
Account acc = new Account(Name = 'Test');  
insert acc;
```

This only tests one record.

Better:

apex

```
List<Account> accounts = new List<Account>();  
  
for (Integer i = 0; i < 200; i++) {  
  
    accounts.add(  
        new Account(  
            Name = 'Test ' + i  
        )  
    );  
}  
  
Test.startTest();  
  
insert accounts;  
  
Test.stopTest();
```

Why 200?

Because Apex trigger processing commonly operates in batches, and testing multiple records helps reveal non-bulkified logic.

Salesforce's own best-practice guidance specifically calls for tests that cover bulk scenarios.  Develo...

30. Why bulk testing matters

Imagine someone writes:

apex

```
for (Account acc : Trigger.new) {  
  
    Contact c = [  
        SELECT Id  
        FROM Contact  
        WHERE AccountId = :acc.Id  
        LIMIT 1  
    ];  
}
```

This puts SOQL inside a loop.

One Account:

1 SOQL

200 Accounts:

200 SOQL

Governor limit problem.

A proper bulk test can expose this.

31. Positive and Negative Testing

A good test isn't only:

"What happens when everything is correct?"

You should also test:

"What happens when something is wrong?"

Suppose:

apex

```
public static void validateAge(Integer age) {  
  
    if (age < 18) {  
        throw new IllegalArgumentException(  
            'Age must be 18 or above'  
        );  
    }  
}
```

```
}
```

Test valid:

apex

```
Test.startTest();

Calculator.validateAge(25);

Test.stopTest();
```

Test invalid:

apex

```
try {

    Calculator.validateAge(15);

    System.assert(false);

} catch (IllegalArgumentException e) {

    System.assertEquals(
        'Age must be 18 or above',
        e.getMessage()
    );
}
```

This is called **negative testing**.

32. Testing exceptions

Another useful pattern:

apex

```
try {

    // Code expected to fail

    System.assert(false, 'Exception was expected');

} catch (Exception e) {

    System.assert(true);
}
```

Why:

If the exception isn't thrown:

apex

```
System.assert(false);
```

fails the test.

33. Testing validation rules / DML errors

Suppose Account has a validation rule requiring:

Annual Revenue > 0

You can test:

apex

```
Account acc = new Account(  
    Name = 'Test'  
);  
  
Database.SaveResult result =  
    Database.insert(acc, false);  
  
System.assertEquals(  
    false,  
    result.isSuccess()  
);
```

This is useful when you expect the database operation to fail.

34. `insert` VS `Database.insert`

You should understand this distinction.

Normal DML

apex

```
insert acc;
```

If there is an error, an exception is thrown.

Database method

apex

```
Database.insert(acc, false);
```

The `false` means:

| Allow partial success.

You receive a `Database.SaveResult`.

Useful when testing error-handling logic.

35. Testing Salesforce Data — The basic pattern

Memorize this:

apex

```
// 1. Create data
Account acc = new Account(
    Name = 'Test Account'
);

// 2. Insert data
insert acc;

// 3. Perform operation
acc.Name = 'Updated Account';

Test.startTest();

update acc;

Test.stopTest();

// 4. Query database
Account result = [
    SELECT Id, Name
    FROM Account
    WHERE Id = :acc.Id
];

// 5. Assert
System.assertEquals(
```

```
'Updated Account',  
result.Name  
);
```

This pattern appears everywhere.

36. Now the important part: Mock Testing

This is different from normal unit testing.

Suppose your Apex code talks to an external API.

Example:

```
Salesforce  
  ↓  
REST API  
  ↓  
External website  
  ↓  
Response
```

During a test, you **don't want to actually call the external service**.

Why?

Because:

- internet/service may be unavailable
- external service may be slow
- response may change
- tests should be deterministic
- Salesforce doesn't allow normal HTTP callouts during tests

So we create a **fake response**.

That's mocking.

Salesforce provides `HttpCalloutMock` for testing HTTP callouts and `WebServiceMock` for SOAP web services. [Develo...](#)

37. REST example — Production code

Suppose:

apex

```
public class WeatherService {  
  
    public static String getWeather() {  
  
        Http http = new Http();  
  
        HttpRequest request = new HttpRequest();  
  
        request.setEndpoint(  
            'https://example.com/weather'  
        );  
  
        request.setMethod('GET');  
  
        HttpResponse response =  
            http.send(request);  
  
        return response.getBody();  
    }  
}
```

Normally:

```
Apex  
↓  
HttpRequest  
↓  
External API  
↓  
HttpResponse  
↓  
getBody()
```

But test shouldn't actually call the API.

38. Create Mock class

apex

```
@isTest  
global class WeatherMock  
    implements HttpCalloutMock {  
  
    global HttpResponse respond(  
        HttpRequest request  
    ) {
```

```
        HttpResponse response =  
            new HttpResponse();  
  
        response.getStatusCode(200);  
  
        response.setBody(  
            '{"temperature":30}'  
        );  
  
        return response;  
    }  
}
```

Let's break this down.

```
implements HttpCalloutMock
```

apex

```
implements HttpCalloutMock
```

means:

This class knows how to provide a fake HTTP response.

```
respond()
```

apex

```
global HttpResponse respond(  
    HttpRequest request  
)
```

Salesforce calls this method instead of the real external API.

Create fake response

apex

```
HttpResponse response =  
    new HttpResponse();
```

Creates our fake response.

Status code

apex

```
response.setStatusCode(200);
```

Pretends API returned:

HTTP 200 OK

Fake body

apex

```
response.setBody(  
    '{"temperature":30}'  
);
```

We're pretending the external API responded:

JSON

```
{  
    "temperature": 30  
}
```

Return it

apex

```
return response;
```

Now Salesforce receives our fake response.

39. Test the REST callout

apex

```
@isTest
private class WeatherServiceTest {

    @isTest
    static void testWeather() {

        Test.setMock(
            HttpCalloutMock.class,
            new WeatherMock()
        );

        Test.startTest();

        String result =
            WeatherService.getWeather();

        Test.stopTest();

        System.assertEquals(
            '{"temperature":30}',
            result
        );
    }
}
```

40. Understand `Test.setMock()`

This line is critical:

apex

```
Test.setMock(
    HttpCalloutMock.class,
    new WeatherMock()
);
```

It tells Salesforce:

"Whenever this test performs an HTTP callout, don't go to the real internet. Use my `WeatherMock` class instead."

So:

Real world:

Apex → Internet → API

Test:

Apex → WeatherMock → Fake Response

This is the heart of mock testing.

41. REST mock flow

Memorize this diagram:

Production code

↓

HttpRequest

↓

http.send()

↓

External API

↓

HttpResponse

Test:

Test

↓

Test.setMock()

↓

WeatherMock

↓

Fake HttpResponse

↓

Production code receives fake response

↓

Assert

42. Testing failure responses

Good developers don't only test:

200 OK

Test:

400
401
404
500

For example:

apex

```
response.getStatusCode(500);
```

Then test whether your Apex correctly handles the server error.

This is what interviewers mean when they ask:

"How would you test different API responses?"

Answer:

Use `HttpCalloutMock` to return different simulated HTTP responses.

43. SOAP Mock Testing

You were recently working on SOAP callouts, so this is particularly important for you.

For SOAP, you generally use:

apex

```
WebServiceMock
```

instead of:

apex

```
HttpCalloutMock
```

Conceptually:

```
REST
```

```
↓
```

```
HttpCalloutMock
```

```
SOAP
```



Salesforce's testing framework supports both approaches. ⚡ Develo...

44. What you should remember for interviews

If interviewer asks:

"What is Apex testing?"

Say:

Apex testing is the process of writing test methods that execute Apex code and verify the expected behavior using assertions. Tests also provide code coverage required for deploying Apex.

"What is unit testing?"

Unit testing tests a small unit of functionality, such as an Apex method, independently from the rest of the application.

"What is test data?"

Test data is data created specifically within the test context to exercise Apex logic. We generally create our own data instead of depending on existing organization data.

"What is `@isTest`?"

`@isTest` identifies Apex classes or methods used for testing.

"What is `@testSetup`?"

`@testSetup` is used to create common test data once so that multiple test methods can reuse it.

"What is `Test.startTest()`?"

It marks the beginning of the code under test, provides a fresh governor-limit context, and helps control asynchronous execution.

"What is `Test.stopTest()`?"

It marks the end of the test execution block and causes queued asynchronous operations to execute during the test.

"How do you test a trigger?"

Create the appropriate Salesforce record, perform the DML operation that fires the trigger, then query the resulting data and use assertions to verify the trigger's behavior.

"Do we call a trigger directly?"

No.

You perform DML:

```
apex  
  
insert acc;
```

or:

```
apex  
  
update acc;
```

The trigger fires automatically.

"What is mocking?"

Mocking replaces an external dependency with a controlled fake implementation so that the test can run independently of the external service.

"How do you test REST callouts?"

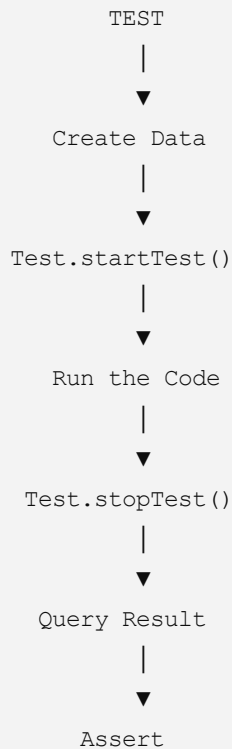
Implement `HttpCalloutMock`, register it using `Test.setMock()`, execute the production method, and assert the returned behavior.

"How do you test SOAP callouts?"

Use `WebServiceMock` to simulate the SOAP response.

45. The MOST important test structure

When you get an interview coding question, think:



That's your basic mental model.

46. Complete interview-style question #1

Question

Create an Apex class that creates an Account and write a test class for it.

Production code

apex

```
public class AccountService {  
  
    public static Account createAccount(  
        String accountName  
    ) {  
  
        Account acc = new Account(  
            Name = accountName  
        );  
  
        insert acc;  
  
        return acc;  
    }  
}
```

Test

apex

```
@isTest  
private class AccountServiceTest {  
  
    @isTest  
    static void testCreateAccount() {  
  
        Test.startTest();  
  
        Account acc =  
            AccountService.createAccount(  
                'Test Company'  
            );  
  
        Test.stopTest();  
  
        System.assertNotEquals(  
            null,  
            acc.Id  
        );  
  
        System.assertEquals(  
            'Test Company',  
            acc.Name  
        );  
    }  
}
```

47. Complete interview-style question #2 — Trigger

Question

Whenever an Account is inserted, create a Task.

Trigger

apex

```
trigger AccountTrigger
on Account (after insert) {

    List<Task> tasks =
        new List<Task>();

    for (Account acc : Trigger.new) {

        Task task = new Task(
            Subject = 'Follow Up',
            WhatId = acc.Id
        );

        tasks.add(task);
    }

    insert tasks;
}
```

Test

apex

```
@isTest
private class AccountTriggerTest {

    @isTest
    static void testTaskCreation() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        Test.startTest();

        insert acc;

        Test.stopTest();
    }
}
```

```

        Task task = [
            SELECT Id, Subject, WhatId
            FROM Task
            WHERE WhatId = :acc.Id
            LIMIT 1
        ];

        System.assertEquals(
            'Follow Up',
            task.Subject
        );

        System.assertEquals(
            acc.Id,
            task.WhatId
        );
    }
}

```

48. Complete interview-style question #3 — Bulk trigger

Question

Write a trigger that creates a Task whenever Accounts are inserted. Make sure it works for multiple Accounts.

Test:

apex

```

@isTest
static void testBulkAccounts() {

    List<Account> accounts =
        new List<Account>();

    for (Integer i = 0; i < 200; i++) {

        accounts.add(
            new Account(
                Name = 'Test Account ' + i
            )
        );
    }

    Test.startTest();

    insert accounts;
}

```

```

Test.stopTest();

List<Task> tasks = [
    SELECT Id, WhatId
    FROM Task
    WHERE WhatId IN :accounts
];

System.assertEquals(
    200,
    tasks.size()
);
}

```

This is a much better trigger test than testing only one Account.

49. Complete interview-style question #4 — Update trigger

Requirement

When an Account's Industry changes to Technology, update Description.

Trigger:

```

apex

trigger AccountTrigger
on Account (before update) {

    for (Account acc : Trigger.new) {

        Account oldAcc =
            Trigger.oldMap.get(acc.Id);

        if (
            oldAcc.Industry != acc.Industry &&
            acc.Industry == 'Technology'
        ) {
            acc.Description =
                'Technology Company';
        }
    }
}

```

Test:

apex

```
@isTest
static void testIndustryChange() {

    Account acc = new Account(
        Name = 'Test Account',
        Industry = 'Finance'
    );

    insert acc;

    acc.Industry = 'Technology';

    Test.startTest();

    update acc;

    Test.stopTest();

    System.assertEquals(
        'Technology Company',
        acc.Description
    );
}
```

50. What does code coverage actually mean?

Suppose your class is:

apex

```
public class TestClass {

    public static String check(Integer number) {

        if (number > 10) {
            return 'Large';
        }

        return 'Small';
    }
}
```

If your test only does:

apex

```
TestClass.check(20);
```

you execute:

```
if condition
  ↓
Large
```

But you never execute:

```
return 'Small';
```

So part of the code isn't covered.

You should test both:

```
apex
TestClass.check(20);
```

and:

```
apex
TestClass.check(5);
```

This is called **branch coverage** in practical terms.

51. Don't chase 100% blindly

This is something I especially want you to remember because you were just dealing with a **6/13 coverage issue**.

Don't think:

"I need to somehow make Salesforce show 100%."

Think:

"I need to execute every meaningful scenario."

For:

```
apex
```

```
if (condition) {  
    // A  
} else {  
    // B  
}
```

you should normally test:

```
condition = TRUE  
condition = FALSE
```

Then the coverage naturally improves.

Salesforce explicitly recommends covering use cases—including positive, negative, and bulk cases—rather than focusing only on the percentage.  Develo...

52. Your Apex Testing Cheat Sheet

Concept	Meaning
<code>@isTest</code>	Marks test class/method
Unit test	Tests a small piece of functionality
Test data	Data created specifically for tests
<code>@testSetup</code>	Creates reusable test data
<code>System.assert()</code>	Checks true/false
<code>assertEquals()</code>	Expected = Actual
<code>assertNotEquals()</code>	Expected ≠ Actual
<code>Test.startTest()</code>	Starts code-under-test context
<code>Test.stopTest()</code>	Ends test context / executes async work
Trigger test	Perform DML and let trigger fire
Bulk test	Test many records
Positive test	Valid scenario
Negative test	Invalid/error scenario

Concept	Meaning
<code>SeeAllData=true</code>	Gives access to org data; generally avoid
Mock	Fake external dependency
<code>HttpCalloutMock</code>	Mock REST/HTTP callout
<code>WebServiceMock</code>	Mock SOAP callout
<code>Test.setMock()</code>	Registers mock implementation
Code coverage	Percentage of Apex code executed

53. The five patterns I want you to memorize

Pattern 1 — Simple Apex method

```
apex

Test.startTest();

    Result result = MyClass.myMethod(input);

Test.stopTest();

System.assertEquals(expected, result);
```

Pattern 2 — Database code

```
apex

// Arrange
Account acc = new Account(
    Name = 'Test'
);

// Act
Test.startTest();

insert acc;

Test.stopTest();
```

```
// Assert
System.assertNotEquals(
    null,
    acc.Id
);
```

Pattern 3 — Trigger

apex

```
// Create record

Account acc = new Account(
    Name = 'Test'
);

// Fire trigger

Test.startTest();

insert acc;

Test.stopTest();

// Check trigger result

Task t = [
    SELECT Id
    FROM Task
    WHERE WhatId = :acc.Id
    LIMIT 1
];

System.assertNotEquals(
    null,
    t.Id
);
```

Pattern 4 — Update trigger

apex

```
Account acc = new Account(
    Name = 'Old'
);
```

```
insert acc;

acc.Name = 'New';

Test.startTest();

update acc;

Test.stopTest();

Account result = [
    SELECT Name
    FROM Account
    WHERE Id = :acc.Id
];

System.assertEquals(
    'New',
    result.Name
);
```

Pattern 5 — REST Mock

```
apex

Test.setMock(
    HttpCalloutMock.class,
    new MyMock()
);

Test.startTest();

String result =
    MyService.callAPI();

Test.stopTest();

System.assertEquals(
    expected,
    result
);
```

54. How I recommend you learn this for interviews

Don't try to memorize 50 different test classes.

Learn this progression:

LEVEL 1

Simple method

↓

assertEquals

LEVEL 2

Apex + Salesforce records

↓

insert / update / delete

↓

SOQL

↓

assert

LEVEL 3

Triggers

↓

insert/update/delete

↓

Trigger.new / Trigger.old

↓

query result

↓

assert

LEVEL 4

Bulk testing

↓

List<SObject>

↓

100-200 records

↓

assert collection results

LEVEL 5

Positive + negative testing

↓

Success scenario

↓

Failure scenario

↓

Exception testing

LEVEL 6

@testSetup

↓

Reusable test data

LEVEL 7

```
Async Apex
  ↓
startTest()
  ↓
stopTest()

LEVEL 8
REST callout
  ↓
HttpCalloutMock

LEVEL 9
SOAP callout
  ↓
WebServiceMock
```

If you understand these **nine levels**, you're in a very strong position for Apex testing questions.

One final mental model

Whenever an interviewer gives you an Apex testing question, ask yourself **five questions**:

1. What am I testing?

```
Class?
Method?
Trigger?
Callout?
Async Apex?
```

2. What data does it need?

```
Account?
Contact?
Opportunity?
Custom object?
```

3. What action causes the code to execute?

```
Method call?
insert?
update?
delete?
API call?
```

4. What should happen?

Record created?
Field updated?
Task created?
Exception?
API response?

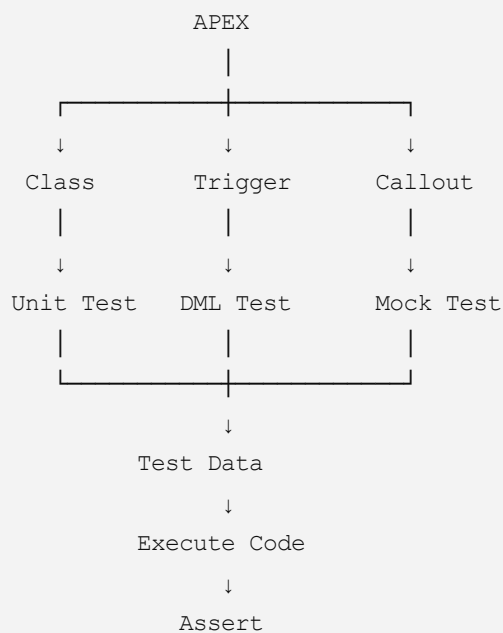
5. How do I prove it?

apex

```
System.assertEquals(...)
System.assertNotEquals(...)
System.assert(...)
```

That thought process is much more valuable than memorizing test-class templates.

And for the topics you've been studying recently, the connection is:



This is the foundation. Salesforce's current developer documentation also emphasizes that tests are required for Apex deployment/package scenarios and that every trigger needs test coverage. ⚡ Developer +1

Next, the most useful way to make this stick would be to practice it as an interview course: start with **Unit Testing** → **Test Data** → `@testSetup` → **Trigger Testing** → **bulk/positive/negative testing** → **REST mock** → **SOAP mock**, with each topic followed by 3–5 coding questions. That will get you from "I understand the explanation" to "I can actually write the test class myself."

List down all the methods and keywords, everything including is, test, test, run, test, stop. And also there are many more methods and keywords or, like, specific type of coding lines or phrases we write regarding testing. So list down all of them and tell me what do they do, why do we use them, and what are the usage of those specific type of, like, coding syntax. What is the use of it? And give me examples using them, so that if the interviewer asks me that why have you used a syntax, or when do we use this syntax, I can answer that.

Yes. For Apex testing, there is a **core set of annotations, methods, keywords, and syntax patterns** that you should know for interviews. I'll organize them by purpose rather than just giving you a random list.

Apex Testing — Complete Syntax & Keyword Cheat Sheet

Think of Apex testing as:

```
TEST CLASS
  ↓
Create Test Data
  ↓
Start Test Context
  ↓
Execute Code
  ↓
Stop Test Context
  ↓
Check Result with Assertions
```

And around that flow, Salesforce gives us several special testing tools.

1. @isTest

This is the most fundamental testing annotation.

apex

```
@isTest
private class AccountTest {
```

```
}
```

What does it do?

It tells Salesforce:

"This class is specifically for testing."

You can also use it on a method:

apex

```
@isTest
static void testAccountCreation() {
}
```

Why use it?

Because Salesforce needs to distinguish test code from production code.

Interview answer

`@isTest` marks an Apex class or method as test code. It tells Salesforce that the code is intended for testing and enables it to run in the test context.

2. `@TestSetup`

Used to create **common test data** that can be reused by multiple test methods.

apex

```
@testSetup
static void createTestData() {

    Account acc = new Account(
        Name = 'Test Account'
    );

    insert acc;
}
```

Then:

apex

```
@isTest
static void testOne() {
```

```
Account acc = [  
    SELECT Id, Name  
    FROM Account  
    LIMIT 1  
];  
}
```

And another test:

```
apex  
  
@isTest  
static void testTwo() {  
    Account acc = [  
        SELECT Id, Name  
        FROM Account  
        LIMIT 1  
    ];  
}
```

Why use it?

Instead of creating the same Account, Contact, etc. repeatedly in every test method.

Interview answer

`@TestSetup` creates common test data once for a test class so that multiple test methods can use that data.

3. `Test.startTest()`

Very important.

```
apex  
  
Test.startTest();
```

It marks the beginning of the actual operation you're testing.

Example:

```
apex  
  
Account acc = new Account(  
    Name = 'Test'  
);
```

```
insert acc;

Test.startTest();

MyClass.processAccount(acc.Id);

Test.stopTest();
```

Notice:

```
Create data
    ↓
Test.startTest()
    ↓
Code being tested
    ↓
Test.stopTest()
```

Why use it?

Primarily:

1. Gives the code under test a fresh governor-limit context.
2. Helps control asynchronous Apex execution.
3. Clearly separates test-data preparation from the actual test execution.

Interview answer

`Test.startTest()` marks the beginning of the test execution context and provides a fresh set of governor limits for the code being tested.

4. `Test.stopTest()`

The partner of `startTest()`.

apex

```
Test.stopTest();
```

Example:

```
apex

Test.startTest();

MyClass.processAccount();
```

Why is it important?

Especially for asynchronous Apex.

For example:

```
Future
Queueable
Batch
Scheduled Apex
```

When asynchronous work is queued between `startTest()` and `stopTest()`, Salesforce executes it synchronously at `stopTest()` in the test context, subject to the rules for that async mechanism.

Interview answer

`Test.stopTest()` marks the end of the test execution context and causes asynchronous operations started during the test context to execute before the test continues.

5. `Test.startTest()` + `Test.stopTest()`

Think of them as a pair.

apex

```
Test.startTest();

// Code under test

Test.stopTest();
```

Typical structure

apex

```
@isTest
static void testSomething() {

    // Arrange
    Account acc = new Account(
        Name = 'Test'
    );
```

```
insert acc;

// Act
Test.startTest();

MyClass.doSomething(acc.Id);

Test.stopTest();

// Assert
System.assertEquals(...);
}
```

6. `System.assertEquals()`

One of your most important testing methods.

apex

```
System.assertEquals(expected, actual);
```

Example:

apex

```
Integer result = 10 + 20;

System.assertEquals(
    30,
    result
);
```

Meaning:

```
Expected = 30
Actual   = result
```

If:

```
result = 30
```

PASS.

If:

```
result = 25
```

FAIL.

7. `System.assertNotEquals()`

Syntax:

apex

```
System.assertNotEquals(expected, actual);
```

Example:

apex

```
System.assertNotEquals(  
    null,  
    acc.Id  
);
```

Meaning:

| Account Id should NOT be null.

Very common after an insert.

8. `System.assert()`

Used when you want to check a Boolean condition.

apex

```
System.assert(condition);
```

Example:

apex

```
Integer age = 25;  
  
System.assert(age >= 18);
```

The test passes because:

```
25 >= 18  
TRUE
```

Another:

```
apex  
  
System.assert(  
    acc.Name == 'Test Account'  
);
```

9. Assertion with a message

You can provide a failure message.

```
apex  
  
System.assertEquals(  
    100,  
    result,  
    'The result should be 100'  
);
```

If it fails, the message helps identify the problem.

Interview point

You can say:

| The optional message makes test failures easier to understand and debug.

10. `Test.setMock()`

This is extremely important for callout testing.

```
apex  
  
Test.setMock(  
    HttpCalloutMock.class,  
    new MyMock()  
);
```

It tells Salesforce:

"When the production code tries to make this HTTP callout, use my fake response instead."

Example:

apex

```
Test.setMock(  
    HttpCalloutMock.class,  
    new WeatherMock()  
);
```

Then:

apex

```
Test.startTest();  
  
WeatherService.getWeather();  
  
Test.stopTest();
```

11. HttpCalloutMock

Used to create a fake HTTP response for REST/HTTP callouts.

apex

```
@isTest  
global class MyMock implements HttpCalloutMock {  
  
    global HttpResponse respond(  
        HttpRequest request  
    ) {  
  
        HttpResponse response =  
            new HttpResponse();  
  
        response.setStatusCode(200);  
  
        response.setBody('Success');  
  
        return response;  
    }  
}
```

The important phrase is:

apex

```
implements HttpCalloutMock
```

Why?

Because Salesforce doesn't want your test to actually call an external service.

Instead:

Real:

Apex → Internet → External API

Test:

Apex → Mock → Fake response

12. `HttpResponse`

Used to construct the fake HTTP response.

apex

```
HttpResponse response = new HttpResponse();
```

Then:

apex

```
response.getStatusCode(200);
```

and:

apex

```
response.setBody('Success');
```

and finally:

apex

```
return response;
```

13. `setStatusCode()`

Used in an HTTP mock to specify the fake HTTP status.

apex

```
response.setStatusCode(200);
```

Common examples:

```
200 → Success
201 → Created
400 → Bad Request
401 → Unauthorized
404 → Not Found
500 → Server Error
```

Example:

apex

```
response.setStatusCode(500);
```

This allows you to test:

"What happens if the external API gives us a server error?"

14. `setBody()`

Sets the fake API response body.

apex

```
response.setBody(
    '{"name": "John", "age": 25}'
);
```

Your production code then receives this as though it came from the real API.

15. `HttpRequest`

Represents an HTTP request.

You might see it in production code:

apex

```
HttpRequest request = new HttpRequest();
```

Then:

apex

```
request.setEndpoint(  
    'https://example.com/api'  
);  
  
request.setMethod('GET');
```

In a mock, you can inspect the incoming request.

For example:

apex

```
global HttpResponse respond(  
    HttpRequest request  
) {  
  
    System.debug(request.getEndpoint());  
  
    ...  
}
```

This lets you verify what the production code attempted to send.

16. `HttpCalloutMock.respond()`

This method is the heart of an HTTP mock.

apex

```
global HttpResponse respond(  
    HttpRequest request  
)
```

Salesforce calls this method when the code under test makes the HTTP callout.

You return:

apex

HttpResponse

17. WebServiceMock

This is the SOAP equivalent of `HttpCalloutMock`.

REST:

apex

HttpCalloutMock

SOAP:

apex

WebServiceMock

You use it when testing generated SOAP web-service callouts.

Interview question:

What's the difference between `HttpCalloutMock` and `WebServiceMock`?

Answer:

`HttpCalloutMock` is used to simulate HTTP/REST callout responses, while `WebServiceMock` is used to simulate SOAP web-service callouts.

18. `Test.isRunningTest()`

This is a special method that tells your Apex code whether it's currently executing inside a test.

apex

`Test.isRunningTest()`

It returns:

```
true
```

during tests.

Example:

```
apex

if (Test.isRunningTest()) {
    // test-specific behavior
}
```

But be careful

Don't use it simply to cheat code coverage.

Bad idea:

```
apex

if (Test.isRunningTest()) {
    return;
}
```

This can make your tests pass without actually testing the important production behavior.

Interview answer

`Test.isRunningTest()` returns true when the code is executing in a test context. It can be useful when specific behavior must differ in tests, but it should not be used merely to bypass production logic or artificially increase coverage.

19. `Test.getCurrentMethodName()`

This can retrieve the name of the currently executing test method in a test context.

```
apex

Test.getCurrentMethodName();
```

It's less commonly used in everyday tests, but useful for test infrastructure/debugging.

You generally don't need to memorize it as strongly as:

```
startTest
stopTest
setMock
isRunningTest
```

20. `Test.getRunId()`

Returns the current test run's ID.

apex

```
Id runId = Test.getRunId();
```

Useful for test infrastructure/debugging.

Again, this is a **secondary** testing API, not something you'll normally use in basic test classes.

21. `Test.loadData()`

Very useful when your test data is stored in a static resource.

Syntax:

apex

```
List<Account> accounts =
    Test.loadData(
        Account.sObjectType,
        'AccountData'
    );
```

Conceptually:

```
CSV/static resource
    ↓
Test.loadData()
    ↓
Salesforce test records
```

When would you use it?

When you have a large amount of structured test data.

Instead of:

apex

```
new Account (...)  
new Account (...)  
new Account (...)  
new Account (...)  
...
```

you can load predefined data.

22. `Test.getStandardPricebookId()`

This one is extremely useful when testing Products/Pricebooks/Opportunity Products.

apex

```
Id pricebookId =  
    Test.getStandardPricebookId();
```

Why?

Because the Standard Price Book is special in Salesforce.

If your test needs it, use the test method rather than relying on an existing org record.

Example:

apex

```
Id pricebookId =  
    Test.getStandardPricebookId();  
  
Pricebook2 pb = [  
    SELECT Id, IsActive  
    FROM Pricebook2  
    WHERE Id = :pricebookId  
];
```

23. `Test.setCreatedDate()`

Allows you to set a record's CreatedDate in a test.

Example:

apex

```
Account acc = new Account(  
    Name = 'Test Account'  
);  
  
insert acc;  
  
Test.setCreatedDate(  
    acc.Id,  
    DateTime.now().addDays(-10)  
);
```

Useful when your logic depends on record age.

For example:

```
Created more than 7 days ago?
```

You can test that without actually waiting seven days.

24. `Test.setCurrentPage()`

Useful for testing Visualforce pages/controllers.

Example:

apex

```
PageReference page =  
    Page.MyVisualforcePage;  
  
Test.setCurrentPage(page);
```

Then your controller can be tested in the context of that page.

25. `Test.setCurrentPageReference()`

Similar concept:

apex

```
Test.setCurrentPageReference(pageReference);
```

Used when testing code that depends on the current Visualforce page reference.

This is more relevant to **Visualforce testing** than LWC.

26. `Test.getStandardPricebookId()` — remember this one

For interviews, I'd put this in your **special-purpose methods** list:

apex

```
Test.getStandardPricebookId();
```

Use when:

Testing functionality involving the Standard Price Book.

27. `Test.startTest()` is NOT a keyword

This is an important terminology point.

You might call:

apex

```
Test.startTest();
```

a "testing syntax" in casual conversation.

But technically:

Test

is the Apex `Test` class.

and:

```
startTest()
```

is a static method of that class.

Same with:

apex

```
Test.stopTest();
```

So:

```
Test.startTest()  
|  
|   └─ method  
|  
└─ Test class
```

This distinction is useful in interviews.

28. Same idea with `System.assertEquals()`

This:

apex

```
System.assertEquals()
```

is:

```
System  
  ↓  
class  
  ↓  
assertEquals()  
  ↓  
method
```

So when someone says:

"Use the assertEquals keyword"

technically it's a **method**, not a keyword.

29. Actual Apex keywords used in test classes

You should know these:

```
private
```

apex

```
private class AccountTest
```

Restricts visibility.

```
public
```

apex

```
public class MyTest
```

Makes it accessible more broadly.

```
static
```

apex

```
static void testAccount()
```

The method belongs to the class rather than an object instance.

Test methods are typically static.

```
void
```

apex

```
static void testAccount()
```

Means the method doesn't return a value.

```
implements
```

apex

```
implements HttpCalloutMock
```

Means the class implements an interface.

30. Why is `static` used in test methods?

You'll almost always see:

apex

```
@isTest
static void testSomething() {
}
```

The test framework invokes the test method directly on the class, so the method is static.

You don't do:

apex

```
AccountTest test = new AccountTest();
test.testSomething();
```

Instead Salesforce executes the test method itself.

31. `global`

You'll sometimes see:

apex

```
global class MyMock implements HttpCalloutMock
```

and:

apex

```
global HttpResponse respond(...)
```

This is especially common in callout mock implementations because the interface method has the required visibility.

For your normal test classes, you generally don't need:

apex

```
global
```

32. `private class` for tests

Very commonly:

```
apex

@isTest
private class AccountTest {
}
```

Why?

Because the test class doesn't need to be part of your public application API.

It's internal test code.

33. `Database.insert()`

Another very important testing syntax.

Normal DML:

```
apex

insert acc;
```

Database method:

```
apex

Database.insert(acc);
```

But the really useful version is:

```
apex

Database.insert(
    acc,
    false
);
```

34. Why `Database.insert(record, false)` ?

The second parameter is:

`allOrNone`

So:

apex

```
Database.insert(records, false);
```

means:

Allow successful records to be inserted even if some records fail.

Then:

apex

```
Database.SaveResult[]
```

can tell you which records succeeded or failed.

35. `Database.SaveResult`

Example:

apex

```
Database.SaveResult result =  
    Database.insert(acc, false);
```

Then:

apex

```
if (result.isSuccess()) {  
    System.debug('Success');  
}
```

or:

apex

```
if (!result.isSuccess()) {  
    System.debug(result.getErrors());  
}
```

Useful for testing error-handling logic.

36. `Test.stopTest()` and asynchronous Apex

Suppose:

apex

```
public class MyQueueable
    implements Queueable {

    public void execute(
        QueueableContext context
    ) {
        // work
    }
}
```

Test:

apex

```
@isTest
static void testQueueable() {

    Test.startTest();

    System.enqueueJob(
        new MyQueueable()
    );

    Test.stopTest();

    // Verify result
}
```

Why does `stopTest()` matter?

Because that's where Salesforce executes the queued async work in the test context.

37. Testing Future methods

Production:

apex

```
@future
public static void processAccount(Id accountId) {
```

```
// asynchronous work  
}
```

Test:

apex

```
Test.startTest();  
  
MyClass.processAccount(acc.Id);  
  
Test.stopTest();
```

Again:

```
startTest  
  ↓  
call future method  
  ↓  
stopTest  
  ↓  
future executes  
  ↓  
assert result
```

38. Testing Queueable

Production:

apex

```
System.enqueueJob(  
    new MyQueueable()  
);
```

Test:

apex

```
Test.startTest();  
  
System.enqueueJob(  
    new MyQueueable()  
);  
  
Test.stopTest();
```

39. Testing Batch Apex

Suppose:

apex

```
Database.executeBatch(  
    new MyBatch(),  
    200  
);
```

Test:

apex

```
Test.startTest();  
  
Database.executeBatch(  
    new MyBatch(),  
    200  
);  
  
Test.stopTest();
```

Then query the result and assert.

40. Testing Scheduled Apex

If production code schedules a job:

apex

```
System.schedule(  
    'My Job',  
    cronExpression,  
    new MyScheduler()  
);
```

Test:

apex

```
Test.startTest();  
  
System.schedule(  
    'Test Job',
```

```
'0 0 12 * * ?',  
    new MyScheduler()  
);  
  
Test.stopTest();
```

Then verify the expected behavior.

41. `Test.getEventBus().deliver()`

This is a more advanced testing method for Platform Events.

Conceptually:

```
apex  
  
Test.getEventBus().deliver();
```

It can be used to control delivery of platform events during tests.

You don't need this for basic Apex interviews, but know that Salesforce provides test-specific APIs for event-driven code as well.

42. `Test.enableChangeDataCapture()`

Another advanced testing API.

Used when testing Change Data Capture behavior.

You don't need to prioritize this unless your project involves CDC.

43. `Test.isBatch()`

There are additional test/context-related methods for determining execution context.

But for your current interview preparation, don't prioritize obscure methods over the core ones.

44. `Test.isRunningTest()` **VS** `@isTest`

This is a very common interview trap.

@isTest

Marks something as test code:

apex

```
@isTest
static void testSomething() {
}
```

Test.isRunningTest()

Checks whether code is currently running inside a test:

apex

```
if (Test.isRunningTest()) {
}
```

So:

```
@isTest
↓
"I'm declaring test code."

Test.isRunningTest()
↓
"Am I currently executing inside a test?"
```

45. SeeAllData=true

Syntax:

apex

```
@isTest(SeeAllData=true)
```

This allows the test to access existing organization data.

Example:

apex

```
@isTest(SeeAllData=true)
private class MyTest {
}
```

Why use it?

Only when there is a specific reason to access real org data.

Why avoid it?

Because tests become dependent on the organization's existing records.

Prefer:

```
apex
```

```
@isTest
```

and create the data yourself.

46. `Test.loadData()` vs `@TestSetup`

Very important difference.

```
@TestSetup
```

You create data in Apex:

```
apex
```

```
@testSetup
static void setup() {

    Account acc = new Account(
        Name = 'Test'
    );

    insert acc;
}
```

```
Test.loadData()
```

You load predefined data from a static resource:

```
apex
```

```
Test.loadData(
    Account.sObjectType,
    'AccountData'
);
```

Think:

```
@TestSetup
↓
Create reusable data with Apex

Test.loadData()
↓
Load predefined test dataset
```

47. `Test.setCreatedDate()` vs normal Date field

Suppose your production logic says:

```
apex

if (acc.CreatedDate < Date.today().addDays(-30)) {
    // old account
}
```

You don't want to wait 30 days.

So:

```
apex

Test.setCreatedDate(
    acc.Id,
    DateTime.now().addDays(-31)
);
```

Now your test can simulate an old record.

48. `Test.setCurrentPage()`

Visualforce example:

```
apex

Test.setCurrentPage(
    Page.AccountPage
);
```

Then:

apex

```
AccountController controller =  
    new AccountController();
```

This is used when testing Visualforce controller behavior.

49. Test class naming convention

Not technically required, but very useful.

Production:

```
AccountService
```

Test:

```
AccountServiceTest
```

Trigger:

```
AccountTrigger
```

Test:

```
AccountTriggerTest
```

This immediately tells developers:

Which production component does this test belong to?

50. Test method naming

Again, no single required naming convention, but use descriptive names.

Instead of:

apex

```
static void test1()
```

prefer:

apex

```
static void testAccountCreation()
```

or:

apex

```
static void testAccountCreationWithValidName()
```

or:

apex

```
static void testAccountCreationFailsWithoutName()
```

The test name itself should tell you what scenario it tests.

51. @TestVisible

This one is worth knowing.

Suppose production code has:

apex

```
private static Integer counter = 10;
```

Normally a test cannot directly access it.

You can use:

apex

```
@TestVisible
private static Integer counter = 10;
```

Now test code can access it.

Why?

It exposes otherwise private members **to test code only** without making them generally public.

Example:

apex

```
@TestVisible
private static Integer MAX_LIMIT = 10;
```

Test:

apex

```
System.assertEquals(
    10,
    MyClass.MAX_LIMIT
);
```

Interview answer

`@TestVisible` allows test methods to access private or protected members without changing their production visibility.

But don't use it to avoid testing behavior properly. Prefer testing through the public behavior of the class when possible.

52. `@TestVisible` vs `public`

This is important.

Without:

apex

```
@TestVisible
private static Integer count;
```

the test can't directly access it.

Instead of making:

apex

```
public static Integer count;
```

you can use:

apex

```
@TestVisible
private static Integer count;
```

So the production design remains private.

53. `System.runAs()`

This is a **major interview topic**.

Syntax:

```
apex

System.runAs(user) {

    // Code

}
```

Example:

```
apex

User testUser = ...;

System.runAs(testUser) {

    Account acc = new Account(
        Name = 'Test'
    );

    insert acc;

}
```

What does it do?

It executes the code in the context of a different Salesforce user.

Why?

To test behavior related to:

- sharing
 - user context
 - record visibility
 - different users
-

54. Example of `System.runAs()`

Suppose you have:

```
Manager  
  ↓  
Employee
```

You want to test what the Employee can see.

You can create a test user:

apex

```
User employee = ...;
```

Then:

apex

```
System.runAs(employee) {  
  
    // perform operations  
}
```

Important interview point

`System.runAs()` changes the user context for the test, but it does **not** magically reproduce every aspect of normal production security enforcement. In particular, object/field permissions are not automatically enforced merely because you use `runAs`.

So don't answer:

"runAs enforces all permissions."

That's incorrect.

55. `User` test data

For `System.runAs()`, you generally need a test User.

Example skeleton:

apex

```

Profile p = [
    SELECT Id
    FROM Profile
    WHERE Name = 'Standard User'
    LIMIT 1
];

User u = new User(
    FirstName = 'Test',
    LastName = 'User',
    Email = 'test@example.com',
    Username = 'unique-test-user@example.com',
    Alias = 'tuser',
    TimeZoneSidKey = 'Asia/Kolkata',
    LocaleSidKey = 'en_IN',
    EmailEncodingKey = 'UTF-8',
    LanguageLocaleKey = 'en_US',
    ProfileId = p.Id
);

insert u;

```

Then:

```

apex

System.runAs(u) {

    // test
}

```

For actual projects, you should design user creation carefully because usernames must be unique.

56. Limits methods

Apex tests can also inspect governor limits.

For example:

```

apex

Limits.getQueries()

```

returns how many SOQL queries have been used.

You can compare:

apex

```
Integer queries =  
    Limits.getQueries();
```

There are corresponding methods for other governor limits.

Examples include:

apex

```
Limits.getDmlStatements()  
Limits.getDmlRows()  
Limits.getCpuTime()  
Limits.getHeapSize()
```

These are useful when you're testing or diagnosing governor-limit-sensitive code.

57. `Limits.getQueries()`

Example:

apex

```
Integer before =  
    Limits.getQueries();  
  
MyClass.process();  
  
Integer after =  
    Limits.getQueries();
```

You could inspect how many queries were consumed.

This is more of an advanced performance/testing technique.

58. `Limits.getDmlStatements()`

apex

```
Integer dml =  
    Limits.getDmlStatements();
```

Tells you how many DML statements have been consumed in the current execution context.

59. `Limits.getCpuTime()`

apex

```
Integer cpu =  
    Limits.getCpuTime();
```

Useful for understanding CPU usage.

Again, don't prioritize this over:

```
assert  
startTest  
stopTest  
mock  
testSetup  
runAs
```

60. `Database.SaveResult` testing pattern

Very useful interview pattern:

apex

```
Database.SaveResult result =  
    Database.insert(  
        acc,  
        false  
    );  
  
System.assertEquals(  
    true,  
    result.isSuccess()  
);
```

For failure:

apex

```
System.assertEquals(  
    false,
```

```
result.isSuccess()  
);
```

Then:

apex

```
Database.Error[] errors =  
    result.getErrors();
```

and:

apex

```
System.assert(  
    errors.size() > 0  
);
```

61. Database.Error

When a DML operation fails:

apex

```
Database.Error error =  
    result.getErrors()[0];
```

You can inspect:

apex

```
error.getMessage();
```

and:

apex

```
error.getStatusCode();
```

and:

apex

```
error.getFields();
```

This is useful for testing error handling.

62. Limits + startTest

This is an important concept.

Before:

apex

```
// Test data setup
```

Then:

apex

```
Test.startTest();
```

Your actual code gets a fresh governor-limit context.

So:

Setup operations

↓

```
Test.startTest()
```

↓

Fresh limits for code under test

↓

```
Test.stopTest()
```

That's why you generally shouldn't perform all your test-data creation inside `startTest()` unless you specifically need to.

63. @TestSetup + startTest are NOT the same

Another interview trap.

@TestSetup

Creates reusable data.

apex

```
@testSetup
static void setup() {
}
```

```
Test.startTest()
```

Starts the execution context for the operation you're testing.

apex

```
Test.startTest();
```

So:

```
@TestSetup
    =
Test data preparation

Test.startTest()
    =
Test execution boundary
```

64. @TestVisible + @isTest are NOT the same

apex

```
@isTest
```

means:

| This is test code.

apex

```
@TestVisible
```

means:

| Allow test code to access this otherwise private/protected member.

65. Mock + startTest/stopTest

Typical REST test:

apex

```

@Test
static void testCallout() {

    Test.setMock(
        HttpCalloutMock.class,
        new MyMock()
    );

    Test.startTest();

    String result =
        MyService.callAPI();

    Test.stopTest();

    System.assertEquals(
        'Success',
        result
    );
}

```

The order matters conceptually:

```

Register mock
    ↓
Start test
    ↓
Call production code
    ↓
Stop test
    ↓
Assert

```

66. The complete Apex Testing vocabulary

Here's the list I'd actually memorize.

MUST KNOW

Syntax	What it does
<code>@isTest</code>	Marks test class/method
<code>@TestSetup</code>	Creates reusable test data
<code>Test.startTest()</code>	Starts test execution context

Syntax	What it does
<code>Test.stopTest()</code>	Ends context / executes async work
<code>System.assert()</code>	Tests a Boolean condition
<code>System.assertEquals()</code>	Checks expected vs actual
<code>System.assertNotEquals()</code>	Checks values aren't equal
<code>Test.setMock()</code>	Registers a mock
<code>HttpCalloutMock</code>	Mocks REST/HTTP callouts
<code>WebServiceMock</code>	Mocks SOAP callouts
<code>Test.isRunningTest()</code>	Checks whether code is running in a test
<code>System.runAs()</code>	Executes test code as another user
<code>@TestVisible</code>	Exposes private/protected members to tests

67. SHOULD KNOW

Syntax	Use
<code>Test.loadData()</code>	Load test data from static resource
<code>Test.setCreateDate()</code>	Change CreatedDate in tests
<code>Test.getStandardPricebookId()</code>	Get Standard Price Book ID
<code>Test.setCurrentPage()</code>	Test Visualforce page context
<code>Database.insert(record, false)</code>	Partial-success DML
<code>Database.SaveResult</code>	Inspect DML result
<code>Database.Error</code>	Inspect DML error
<code>Limits.getQueries()</code>	Check SOQL usage
<code>Limits.getDmlStatements()</code>	Check DML usage

Syntax

```
Limits.getCpuTime()
```

Use

Check CPU usage

68. 🟡 ADVANCED / KNOW THE PURPOSE

These aren't the first things I'd memorize:

```
Test.getRunId()  
Test.getCurrentMethodName()  
Test.getEventBus().deliver()  
Test.enableChangeDataCapture()
```

These become relevant for specialized testing scenarios.

69. The syntax patterns you should be able to write from memory

Pattern A — Basic test

```
apex  
  
@isTest  
static void testSomething() {  
  
    Test.startTest();  
  
    // Code under test  
  
    Test.stopTest();  
  
    System.assertEquals(  
        expected,  
        actual  
    );  
}
```

Pattern B — Salesforce data

```
apex
```

```
Account acc = new Account(  
    Name = 'Test'  
);  
  
insert acc;  
  
Test.startTest();  
  
update acc;  
  
Test.stopTest();  
  
Account result = [  
    SELECT Id, Name  
    FROM Account  
    WHERE Id = :acc.Id  
];  
  
System.assertEquals(  
    'Expected',  
    result.Name  
);
```

Pattern C — Trigger

apex

```
Account acc = new Account(  
    Name = 'Test'  
);  
  
Test.startTest();  
  
insert acc;  
  
Test.stopTest();  
  
// Query trigger's result  
  
System.assertEquals(  
    expected,  
    actual  
);
```

Remember:

You don't call the trigger directly. You perform the DML that causes it to fire.

Pattern D — Bulk trigger

apex

```
List<Account> accounts =
    new List<Account>();

for (Integer i = 0; i < 200; i++) {

    accounts.add(
        new Account (
            Name = 'Test ' + i
        )
    );
}

Test.startTest();

insert accounts;

Test.stopTest();

// Query results

System.assertEquals(
    200,
    results.size()
);
```

Pattern E — REST mock

apex

```
Test.setMock(
    HttpCalloutMock.class,
    new MyMock()
);

Test.startTest();

String result =
    MyService.callAPI();

Test.stopTest();

System.assertEquals(
    expected,
```

result

Pattern F — User context

apex

```
User u = ...;

System.runAs(u) {

    // Code being tested

}
```

Pattern G — Async

apex

```
Test.startTest();

System.enqueueJob(
    new MyQueueable()
);

Test.stopTest();

// Assert result
```

70. One thing you should NOT memorize blindly

Don't memorize:

apex

```
Test.startTest();
Test.stopTest();
System.assertEquals();
```

as isolated commands.

Understand **why they exist**.

Think:

```

@Test
↓
"I'm writing a test."

@TestSetup
↓
"I need common test data."

Test.startTest()
↓
"Now execute the actual operation."

Test.stopTest()
↓
"Finish the test context / execute async work."

System.assertEquals()
↓
"Now prove that the expected result happened."

Test.setMock()
↓
"Don't call the real external service."

HttpCalloutMock
↓
"Give the production code a fake HTTP response."

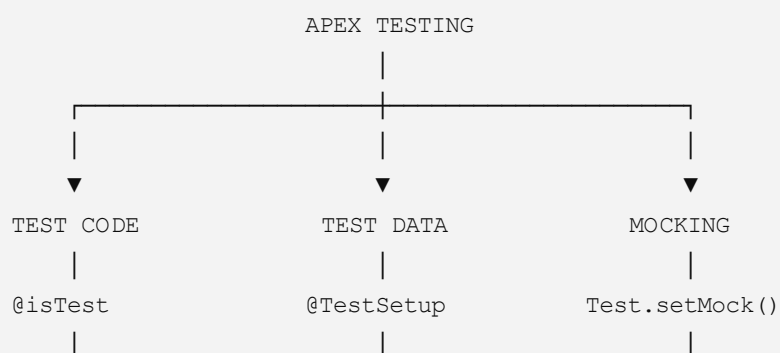
System.runAs()
↓
"Test behavior in another user's context."

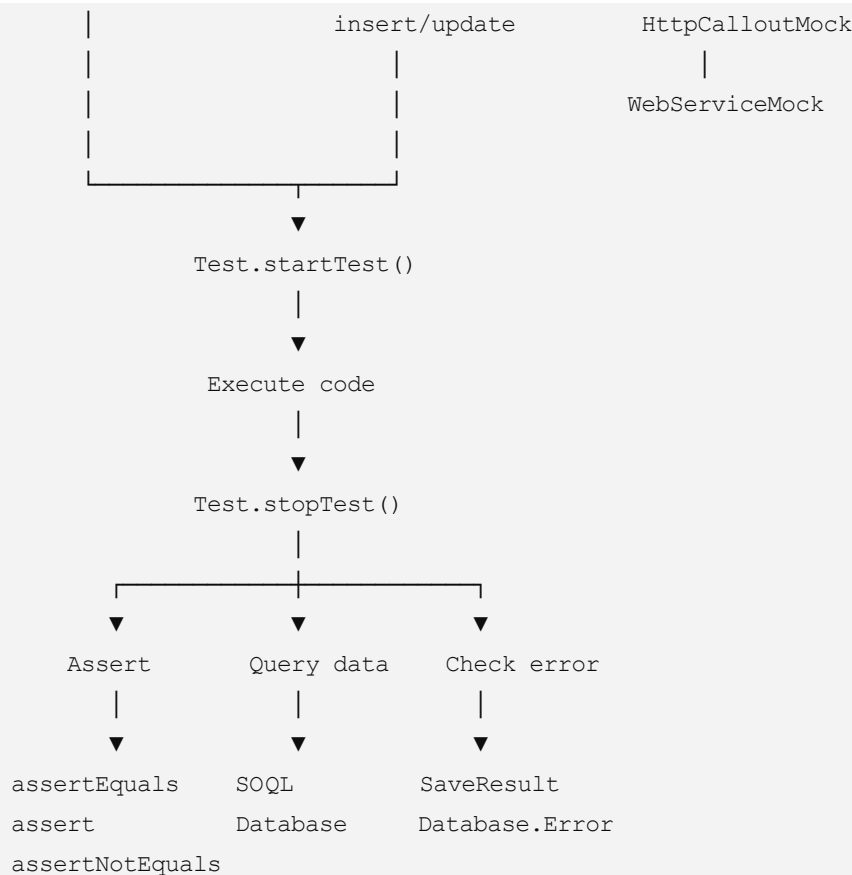
@TestVisible
↓
"Let test code access a private/protected member."

```

If you can explain that chain in an interview, you aren't just memorizing syntax—you understand Apex testing.

The one-page mental map





If you master **this map + the patterns above**, you will be able to reason through most Apex testing interview questions rather than trying to remember an exact answer.

Okay, so give me five questions regarding, you know, any sort of Salesforce activities. It can be certain sort of action on any object, or whatever. Give me count of certain things, like update these things, or anything. Like, you know what? Five different types of questions, and write the exact codes for them, and test code for them as well, so that I can understand that. Like that will do both the works. A, when they'll give me situations and write code on it, I can write the code on it by saying the logic, by making the logic I can write Apex code on it, and also by writing my code I can write the test code on it. Explain. If the corporate is like that, Show more

Exactly. This is the **right way to prepare for an Apex interview**: don't just learn testing syntax separately. You should learn to look at a requirement and immediately think:

What production code would I write? → What scenarios does that code have? → What test data do I need? → What action triggers the code? → What result should

I assert?

Below are **5 deliberately different interview-style problems**. Together they cover:

1. Normal Apex class + DML
2. Trigger + related records
3. SOQL + counting/aggregation
4. Bulk processing + update logic
5. REST callout + Mock testing

For every question I'll show:

Requirement → **Logic** → **Production code** → **Test code** → **Why the test looks like that**
→ **Interview thinking**

Question 1 — Update an Account based on Number of Contacts

Interview question

Whenever a method is called for an Account, count the Contacts belonging to that Account. If the Account has 5 or more Contacts, set its Description to `High Contact Account`; otherwise set it to `Normal Contact Account`.

This is a good question because it tests:

- Apex class
 - SOQL
 - Aggregate/count logic
 - DML
 - Test data
 - Positive + negative testing
-

Step 1 — Think about the logic

We need:

```
Account Id
  ↓
Find Contacts belonging to Account
  ↓
Count Contacts
  ↓
```

Is count >= 5?

↓

YES → High Contact Account

NO → Normal Contact Account

Production code

apex

```
public class AccountContactService {

    public static void updateAccountDescription(Id accountId) {

        Integer contactCount = [
            SELECT COUNT()
            FROM Contact
            WHERE AccountId = :accountId
        ];

        Account acc = [
            SELECT Id, Description
            FROM Account
            WHERE Id = :accountId
            LIMIT 1
        ];

        if (contactCount >= 5) {
            acc.Description = 'High Contact Account';
        } else {
            acc.Description = 'Normal Contact Account';
        }

        update acc;
    }
}
```

Understand the code

1.

apex

```
public class AccountContactService {
```

Our Apex class.

2.

apex

```
public static void updateAccountDescription(Id accountId)
```

We're creating a method.

It receives:

Account Id

and doesn't return anything:

void

3.

apex

```
Integer contactCount = [  
    SELECT COUNT()  
    FROM Contact  
    WHERE AccountId = :accountId  
];
```

This is the important part.

We're asking Salesforce:

How many Contacts belong to this Account?

`COUNT()` gives us the number.

The `:` means:

Use the Apex variable `accountId` inside the SOQL query.

4.

apex

```
Account acc = [  
    SELECT Id, Description  
    FROM Account  
    WHERE Id = :accountId  
    LIMIT 1  
];
```

Get the Account we're going to modify.

5.

apex

```
if (contactCount >= 5)
```

Business condition.

6.

apex

```
acc.Description = 'High Contact Account';
```

Update the field in memory.

7.

apex

```
update acc;
```

Actually save it to Salesforce.

Now the TEST CODE

This is where you should start thinking:

| What scenarios exist?

There are **two branches**:

5 or more Contacts → High
Less than 5 → Normal

So we should test both.

Test 1 — 5 Contacts

apex

```
@isTest
private class AccountContactServiceTest {

    @isTest
    static void testHighContactAccount() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        insert acc;

        List<Contact> contacts = new List<Contact>();

        for (Integer i = 0; i < 5; i++) {

            contacts.add(
                new Contact(
                    FirstName = 'Test',
                    LastName = 'Contact ' + i,
                    AccountId = acc.Id
                )
            );
        }

        insert contacts;

        Test.startTest();

        AccountContactService.updateAccountDescription(
            acc.Id
        );

        Test.stopTest();

        Account result = [
            SELECT Id, Description
```

```

        FROM Account
        WHERE Id = :acc.Id
    ];

    System.assertEquals(
        'High Contact Account',
        result.Description
    );
}
}

```

Why did we create 5 Contacts?

Because the production code says:

```

apex

if (contactCount >= 5)

```

So our test data must specifically satisfy that condition.

This is a key interview skill:

Test data should be designed according to the conditions in the production code.

Test 2 — Less than 5

```

apex

@isTest
static void testNormalContactAccount() {

    Account acc = new Account(
        Name = 'Test Account'
    );

    insert acc;

    Contact con = new Contact(
        FirstName = 'Test',
        LastName = 'Contact',
        AccountId = acc.Id
    );

    insert con;
}

```

```
Test.startTest();

AccountContactService.updateAccountDescription(
    acc.Id
);

Test.stopTest();

Account result = [
    SELECT Id, Description
    FROM Account
    WHERE Id = :acc.Id
];

System.assertEquals(
    'Normal Contact Account',
    result.Description
);
}
```

Now you've covered:

```
contactCount >= 5
+
contactCount < 5
```

So both branches are tested.

What should you say in an interview?

If they ask:

"How would you test this?"

Say:

"The method has two logical branches, so I would create test data for both scenarios. I'd create an Account with five Contacts to test the true condition and an Account with fewer than five Contacts to test the false condition. Then I'd call the method, query the updated Account, and use assertions to verify the Description."

That's a **very strong answer**.

Question 2 — Trigger: Create a Task when an Opportunity is Closed Won

Interview question

Create a trigger that automatically creates a follow-up Task whenever an Opportunity becomes `Closed Won`.

This tests:

- Trigger
- `Trigger.new`
- `Trigger.oldMap`
- Update operation
- Related record creation
- Test trigger by performing DML

First think about the requirement

Important word:

becomes

We don't want:

`Opportunity already Closed Won`

We want:

`Old Stage = Negotiation`
`New Stage = Closed Won`

Therefore we need:

apex

`Trigger.oldMap`

Production trigger

apex

```
trigger OpportunityTrigger on Opportunity (after update) {  
  
    List<Task> tasks = new List<Task>();
```

```

for (Opportunity opp : Trigger.new) {

    Opportunity oldOpp =
        Trigger.oldMap.get(opp.Id);

    if (
        oldOpp.StageName != 'Closed Won' &&
        opp.StageName == 'Closed Won'
    ) {

        Task task = new Task(
            Subject = 'Follow up with Closed Won customer',
            WhatId = opp.Id,
            OwnerId = opp.OwnerId
        );

        tasks.add(task);
    }
}

if (!tasks.isEmpty()) {
    insert tasks;
}
}

```

Understand the logic

Suppose:

```

Old:
Stage = Negotiation

New:
Stage = Closed Won

```

Then:

apex

```
oldOpp.StageName != 'Closed Won'
```

is:

```
TRUE
```

and:

apex

```
opp.StageName == 'Closed Won'
```

is:

TRUE

Therefore:

Create Task

Why `Trigger.oldMap`?

Because we need to know:

| What was the old value?

`Trigger.new` tells us the new version.

`Trigger.oldMap` gives us the previous version indexed by Id.

This is a classic interview question.

Test code

apex

```
@isTest
private class OpportunityTriggerTest {

    @isTest
    static void testClosedWonTaskCreation() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        insert acc;

        Opportunity opp = new Opportunity(
            Name = 'Test Opportunity',
            AccountId = acc.Id,
            StageName = 'Prospecting',
```

```

        CloseDate = Date.today().addDays(30)
    );

    insert opp;

    Test.startTest();

    opp.StageName = 'Closed Won';

    update opp;

    Test.stopTest();

    Task task = [
        SELECT Id, Subject, WhatId
        FROM Task
        WHERE WhatId = :opp.Id
        LIMIT 1
    ];

    System.assertEquals(
        'Follow up with Closed Won customer',
        task.Subject
    );

    System.assertEquals(
        opp.Id,
        task.WhatId
    );
}
}

```

Why does the test first insert the Opportunity?

Because the trigger is:

```

    apex
    after update

```

It only fires when we perform:

```

    apex
    update opp;

```

So:

Create Opportunity

↓

insert

↓

Opportunity exists

↓

Change Stage

↓

update

↓

Trigger fires

Why don't we directly call the trigger?

You don't do:

apex

```
OpportunityTrigger.run();
```

There is no normal direct trigger invocation.

Instead:

apex

```
update opp;
```

causes Salesforce to execute the trigger.

This is extremely important.

Question 3 — Count Opportunities and Update Account

Now let's make the requirement more realistic.

Interview question

For every Account, calculate how many Opportunities are currently Closed Won. If the Account has at least 3 Closed Won Opportunities, update a custom field

`Customer_Status__c` to `VIP` ; otherwise set it to `Regular` .

Assume:

```
Account.Customer_Status__c
```

is a custom picklist/text field.

This tests:

- SOQL
- Aggregate query
- Parent-child relationship
- Business logic
- DML
- Test data

Production code

apex

```
public class AccountOpportunityService {

    public static void updateCustomerStatus(Id accountId) {

        Integer wonCount = [
            SELECT COUNT()
            FROM Opportunity
            WHERE AccountId = :accountId
            AND StageName = 'Closed Won'
        ];

        Account acc = [
            SELECT Id, Customer_Status__c
            FROM Account
            WHERE Id = :accountId
            LIMIT 1
        ];

        if (wonCount >= 3) {
            acc.Customer_Status__c = 'VIP';
        } else {
            acc.Customer_Status__c = 'Regular';
        }

        update acc;
    }
}
```

Test code

apex

```
@isTest
private class AccountOpportunityServiceTest {

    @isTest
    static void testVipAccount() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        insert acc;

        List<Opportunity> opportunities =
            new List<Opportunity>();

        for (Integer i = 0; i < 3; i++) {

            opportunities.add(
                new Opportunity(
                    Name = 'Test Opportunity ' + i,
                    AccountId = acc.Id,
                    StageName = 'Closed Won',
                    CloseDate = Date.today()
                )
            );
        }

        insert opportunities;

        Test.startTest();

        AccountOpportunityService
            .updateCustomerStatus(acc.Id);

        Test.stopTest();

        Account result = [
            SELECT Customer_Status__c
            FROM Account
            WHERE Id = :acc.Id
        ];

        System.assertEquals(
            'VIP',
            result.Customer_Status__c
        );
    }
}
```

```
}
```

Why exactly 3 Opportunities?

Because the requirement says:

apex

```
if (wonCount >= 3)
```

Therefore:

0 → Regular

1 → Regular

2 → Regular

3 → VIP

4 → VIP

We choose exactly **3** because it tests the boundary condition.

That's another interview concept:

Boundary-value testing.

For:

```
>= 3
```

you should ideally test:

2

3

4

because that's where the behavior changes.

Add the negative test

apex

```
@isTest
static void testRegularAccount() {
```

```

Account acc = new Account(
    Name = 'Test Account'
);

insert acc;

Opportunity opp = new Opportunity(
    Name = 'Test Opportunity',
    AccountId = acc.Id,
    StageName = 'Closed Won',
    CloseDate = Date.today()
);

insert opp;

Test.startTest();

AccountOpportunityService
    .updateCustomerStatus(acc.Id);

Test.stopTest();

Account result = [
    SELECT Customer_Status__c
    FROM Account
    WHERE Id = :acc.Id
];

System.assertEquals(
    'Regular',
    result.Customer_Status__c
);
}

```

Now you've tested:

3 → VIP
1 → Regular

Question 4 — Bulk Apex

Now here's a very important corporate/interview scenario.

Question

When Accounts are created, set a custom field `Customer_Type__c` to `Large` if Annual Revenue is greater than ₹10 crore, otherwise set it to `Small`. The solution must work for multiple Accounts at once.

This tests:

- Trigger
 - Bulkification
 - `Trigger.new`
 - No SOQL in loop
 - No DML in loop
 - List processing
 - Bulk test
-

Trigger

apex

```
trigger AccountTrigger on Account (before insert) {

    for (Account acc : Trigger.new) {

        if (
            acc.AnnualRevenue != null &&
            acc.AnnualRevenue > 100000000
        ) {
            acc.Customer_Type__c = 'Large';
        } else {
            acc.Customer_Type__c = 'Small';
        }
    }
}
```

Why is this bulkified?

Look at:

apex

```
for (Account acc : Trigger.new)
```

We're processing all records in the trigger batch.

We aren't doing:

apex

```
insert
```

inside the loop.

We aren't doing:

```
apex
```

```
SOQL
```

inside the loop.

And because it's `before insert`, we don't even need another DML operation.

We simply change:

```
apex
```

```
acc.Customer_Type__c
```

and Salesforce saves the modified value.

Test bulk behavior

```
apex
```

```
@isTest
private class AccountTriggerTest {

    @isTest
    static void testBulkAccountCreation() {

        List<Account> accounts =
            new List<Account>();

        for (Integer i = 0; i < 200; i++) {

            Decimal revenue;

            if (i < 100) {
                revenue = 200000000;
            } else {
                revenue = 500000000;
            }

            accounts.add(
                new Account(
```

```

        Name = 'Test Account ' + i,
        AnnualRevenue = revenue
    )
    );
}

Test.startTest();

insert accounts;

Test.stopTest();

List<Account> results = [
    SELECT Id, Customer_Type__c
    FROM Account
    WHERE Id IN :accounts
];

Integer largeCount = 0;
Integer smallCount = 0;

for (Account acc : results) {

    if (acc.Customer_Type__c == 'Large') {
        largeCount++;
    }

    if (acc.Customer_Type__c == 'Small') {
        smallCount++;
    }
}

System.assertEquals(
    100,
    largeCount
);

System.assertEquals(
    100,
    smallCount
);
}
}

```

Why did we create 200 records?

Because we want to answer:

Does this trigger work when Salesforce processes many records at once?

Not just:

1 Account

but:

200 Accounts

What exactly are we testing?

100 Large
+
100 Small
=
200 total

So we're testing both branches **and** bulk behavior.

This is an excellent corporate-style test.

Question 5 — REST API + Mock Testing

Now let's switch completely.

Interview question

Create an Apex class that calls an external REST API and returns the response. Write a test class without making a real HTTP callout.

This tests:

- HTTP
 - REST callout
 - `HttpRequest`
 - `HttpResponse`
 - `HttpCalloutMock`
 - `Test.setMock()`
 - `startTest()`
 - `stopTest()`
-

Production Apex

apex

```
public class CustomerApiService {  
  
    public static String getCustomer() {  
  
        HttpRequest request =  
            new HttpRequest();  
  
        request.setEndpoint(  
            'https://example.com/customer'  
        );  
  
        request.setMethod('GET');  
  
        Http http = new Http();  
  
        HttpResponse response =  
            http.send(request);  
  
        return response.getBody();  
    }  
}
```

Understand the flow

```
Create HttpRequest  
    ↓  
Set endpoint  
    ↓  
Set HTTP method  
    ↓  
Create Http  
    ↓  
send(request)  
    ↓  
HttpResponse  
    ↓  
getBody()
```

Why can't we simply test this normally?

Because this:

apex

```
http.send(request);
```

tries to make an actual external callout.

Instead we create a fake response.

Mock class

apex

```
@isTest
global class CustomerApiMock
    implements HttpCalloutMock {

    global HttpResponse respond(
        HttpRequest request
    ) {

        HttpResponse response =
            new HttpResponse();

        response.setStatusCode(200);

        response.setBody(
            '{"id":101,"name":"John"}'
        );

        return response;
    }
}
```

Line-by-line

This:

apex

```
implements HttpCalloutMock
```

means:

This class is going to provide a fake HTTP response.

This:

apex

```
global HttpResponse respond(  
    HttpRequest request  
)
```

is the method Salesforce invokes when the code makes the callout.

This:

apex

```
response.setStatusCode(200);
```

pretends the external server returned:

200 OK

This:

apex

```
response.setBody(  
    '{"id":101,"name":"John"}'  
);
```

pretends the API returned that JSON.

Test class

apex

```
@isTest  
private class CustomerApiServiceTest {  
  
    @isTest  
    static void testCustomerApi() {
```

```

    Test.setMock(
        HttpCalloutMock.class,
        new CustomerApiMock()
    );

    Test.startTest();

    String response =
        CustomerApiService.getCustomer();

    Test.stopTest();

    System.assertEquals(
        '{"id":101,"name":"John"}',
        response
    );
}

```

Understand the critical line

apex

```

Test.setMock(
    HttpCalloutMock.class,
    new CustomerApiMock()
);

```

You're effectively telling Salesforce:

"When CustomerApiService calls an HTTP endpoint,

DO NOT actually call the internet.

Instead use CustomerApiMock."

So:

PRODUCTION

Apex

↓

HTTP

↓

External API

↓

Response

Test:

```
TEST
  ↓
Apex
  ↓
HTTP
  ↓
CustomerApiMock
  ↓
Fake Response
```

Now let's test an API failure

A really good interviewer may ask:

"What if the API returns 500?"

Create another mock:

```
apex

@isTest
global class CustomerApiErrorMock
    implements HttpCalloutMock {

    global HttpResponse respond(
        HttpRequest request
    ) {

        HttpResponse response =
            new HttpResponse();

        response.setStatusCode(500);

        response.setBody(
            '{"error": "Server Error"}'
        );

        return response;
    }
}
```

Then:

```
apex
```

```

@Test
static void testCustomerApiError() {

    Test.setMock(
        HttpCalloutMock.class,
        new CustomerApiErrorMock()
    );

    Test.startTest();

    String response =
        CustomerApiService.getCustomer();

    Test.stopTest();

    System.assertEquals(
        '{"error": "Server Error"}',
        response
    );
}

```

This is **negative testing of an external dependency**.

Now look at all 5 questions together

This is the part I really want you to understand.

Question	Production concept	Test concept
1. Contact count	Apex class + SOQL + DML	Create related data + call method + query + assert
2. Closed Won Task	Trigger + old/new values	Insert → update → trigger fires → query Task → assert
3. Won Opportunity count	SOQL + business logic	Create Opportunities → execute → query Account → assert
4. Account classification	Bulk trigger	Create 200 records → insert once → verify all
5. REST API	HTTP callout	HttpCalloutMock + Test.setMock()

The REALLY important part: How do I know what test to write?

This is what you asked about when you said:

"If there's a certain kind of code, what kind of test code do I need to write?"

Use this table.

If production code contains...	Your test should usually...
Normal method	Call the method + assert return/result
<code>insert</code>	Create test data + call code + verify record
<code>update</code>	Create record + modify it + call update + verify
<code>delete</code>	Create record + execute delete + query/verify absence
SOQL	Create matching test data first
<code>if/else</code>	Test both branches
<code>for</code> loop	Test multiple records
Trigger <code>before insert</code>	Insert records and verify modified fields
Trigger <code>after insert</code>	Insert records and query records created by trigger
Trigger <code>before update</code>	Insert first → change field → update
<code>Trigger.old</code> / <code>oldMap</code>	Create old state → modify → update
Related record creation	Query the related record and assert
Exception	Deliberately cause error + verify exception
Future method	<code>startTest()</code> → call future → <code>stopTest()</code> → assert
Queueable	<code>startTest()</code> → enqueue → <code>stopTest()</code> → assert

If production code contains...	Your test should usually...
Batch	<code>startTest()</code> → execute batch → <code>stopTest()</code> → assert
Scheduled Apex	Schedule inside test context → <code>stopTest()</code> → assert
REST callout	<code>HttpCalloutMock</code>
SOAP callout	<code>WebServiceMock</code>
Different API responses	Create different mock classes/responses
User-dependent logic	<code>System.runAs()</code>
Common test data	<code>@TestSetup</code>
Private member that genuinely needs direct testing	<code>@TestVisible</code>

The 7-step thought process for ANY interview question

Suppose tomorrow the interviewer says:

"Whenever a Lead is converted, update something on the Account."

Don't immediately start writing code.

Think:

Step 1 — What event?

Lead conversion

Step 2 — What data?

Lead
Account

Step 3 — What Apex mechanism?

Maybe:

Trigger

Step 4 — What records change?

Account

Step 5 — What scenarios exist?

Conversion succeeds
Conversion doesn't satisfy condition
Multiple records / bulk behavior

Step 6 — What test data do I create?

Lead
Account

Step 7 — What do I assert?

Account field changed correctly

That's how you move from **requirement** → **production code** → **test code**.

One more example of the thinking process

Suppose interviewer says:

"When an Opportunity changes from Negotiation/Review to Closed Won, create a Task."

Your brain should immediately go:

```
"changes from"  
  ↓  
Need old + new value  
  ↓  
Trigger  
  ↓  
after update  
  ↓  
Trigger.oldMap  
  ↓  
Trigger.new
```

```
↓
Create Task
↓
Test:
  Create Opportunity
  ↓
  Insert
  ↓
  Change Stage
  ↓
  Update
  ↓
  Trigger fires
  ↓
  Query Task
  ↓
  Assert
```

That is the skill you need for interviews.

And this is how you should think about code coverage

Don't think:

"How do I get 100%?"

Think:

```
What branches does my code have?
↓
What conditions exist?
↓
What records trigger them?
↓
Have I tested each scenario?
```

For example:

```
apex

if (amount > 100000) {
  // A
} else {
  // B
}
```

Your test should contain:

```
Test 1:
amount = 150000
      ↓
A

Test 2:
amount = 50000
      ↓
B
```

If you have:

apex

```
if (oldStage != 'Closed Won'
    && newStage == 'Closed Won')
```

your test data must specifically create:

```
Old = something else
New = Closed Won
```

That's how you **naturally get coverage** instead of trying to manipulate coverage.

Your interview-level checklist

When you see an Apex requirement, ask:

1. What Salesforce object is involved?
2. What event/action happens? insert / update / delete / callout
3. Class or Trigger?
4. What data do I need?
5. Are there if/else branches?
6. Does it need old + new values?
7. Does it need to work in bulk?
8. Does it make an external callout?
9. What should the final result be?

10. What assertion proves that result?

If you can answer those 10 questions, you can usually derive **both the Apex code and its test class** instead of memorizing them.

One important next step: take these five examples and try writing the **test classes yourself without looking**. Especially try Question 2 and Question 4 from scratch. Those two will force you to understand the difference between **testing a normal Apex class and testing a trigger**, and between **single-record and bulk testing**.

ChatGPT can make mistakes. Check important info.